Correspondence Problem

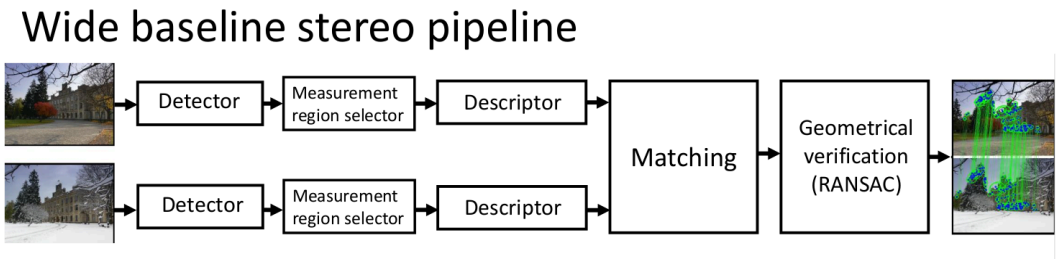
Searching for correspondences is a fundamental task in computer vision. The goal is to find corresponding parts (patches) of a scene in two or more images. Finding correspondences on a pixel level is often ill-posed, and instead correspondences for selected larger image portions (patches) are sought. This comprises of first detecting, in each image, the local features (a.k.a. interest points, distinguished regions, covariant regions) i.e. **regions** detectable in other images of the same scene with high degree of geometric and photometric invariance. Correspondences are then sought between these local features.

For each feature, the exact position and a description of the feature neighborhood is saved. The description can be a vector of pixel intensities, histogram of intensities, histogram of gradients, etc. The choice of a descriptor influences the discrimination and invariance of the feature. Roughly speaking, local feature descriptor is a vector, which characterises the image function in a small, well localized neighborhood of a feature point.

When establishing correspondences, for each feature in one image we look for features in the second image that have a similar description. We call these correspondences "tentative", as some of them are usually wrong – they do not associate the same points in the scene. This happens because of noise, repeated structures in the scene, occlusions or inaccuracies of models of geometric and photometric transformation.

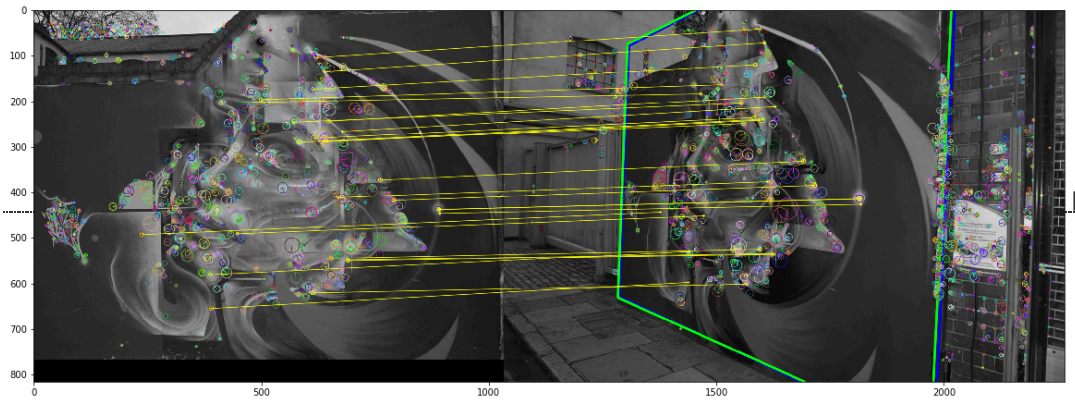
As a next step we need a robust algorithm, which (based on the knowledge of the image-to-image transformation model) will choose the largest subset of correspondences which are all consistent with the model. Such correspondences are called „verified" or „inliers" (with the model). An algorithm widely used for this purpose is called RANSAC (RANdom Sampling Consensus) and will be introduced it in the last section.

This can be implemented with a few lines of code using OpenCV library (see this notebook[1]). We omit here data load and reshape for clarity:



[2]

```
det = cv2.AKAZE_create()
kps1, descs1 = det.detectAndCompute(img1,None)
kps2, descs2 = det.detectAndCompute(img2,None)
matcher = cv2.DescriptorMatcher_create(cv2.DescriptorMatcher_BRUTEFORCE_HAMMING)
knn_matches = matcher.knnMatch(descriptors1, descriptors2, 2)
ratio_thresh = 0.8
tentative_matches = []
for m,n in knn_matches:
   if m.distance < 0.8 * n.distance:
       tentative_matches.append(m)
src_pts, dst_pts = split_pts(tentative_matches, kps1, kps2)
H, inlier_mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC,1.0)
```



[3]

In this course we will implement all these components from scratch using PyTorch and numpy

1. Feature detection

Feature detection is the first step in the process. We will implement the Harris detector.

Harris detector

Harris detector detects locations in which the local autocorrelation function is changing in all directions (therefore, the image gradient direction is changing there). For this reason, it is often called a corner detector. It computes the autocorrelation matrix, $\mathbf{M}(x, y; \sigma_d; \sigma_i)$:

$$\mathbf{M}(x, y; \sigma_d, \sigma_i) = G(x, y; \sigma_i) * \begin{bmatrix} D_x^2(x, y; \sigma_d) & D_x D_y(x, y; \sigma_d) \\ D_x D_y(x, y; \sigma_d) & D_y^2(x, y; \sigma_d) \end{bmatrix}$$

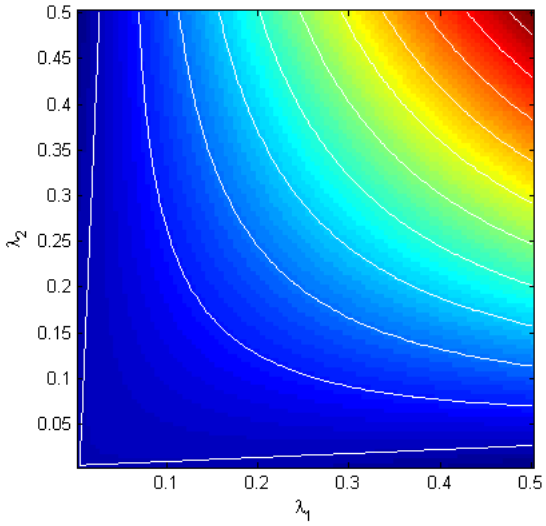
where * is the convolution operator. This is weighted averaging (windowing function $G(x, y; \sigma_i)$) of outer products of gradients. If both eigenvalues of $\mathbf{M}(x, y; \sigma_d; \sigma_i)$ are large, and are of comparable value then the pattern inside the windowing function has stable position and we want to use it for matching. Harris and Stephens[4] select such points based on *corner response*:

$$R(x, y) = \det(\mathbf{M}) - \alpha \mathrm{trace}^2(\mathbf{M}).$$

where the advantage is that eigenvalues λ_1, λ_2 of matrix $\mathbf{M}$ do not have to be explicitly computed because

$$\begin{aligned} \det(\mathbf{M}) &= \lambda_1 \lambda_2 \\ \mathrm{trace}(\mathbf{M}) &= \lambda_1 + \lambda_2. \end{aligned}$$

Note however that explicit computation of eigenvalues would not be a problem with todays computers (how would you do it?)



[5]

Graph of $R(x, y)$ for $\alpha = 0.04$. White curves are isocontours for values $0, 0.02, 0.04, 0.06, \ldots, 0.18$

The image shows the function $R(x, y)$ for $\alpha = 0.04$, where white lines show iso-contours with values 0, 0.02, 0.04, 0.06, ..., 0.18. When we want to have values greater than a threshold, we want the smaller of the two eigenvalues to be greater than the threshold. We also want to have the value of the greater eigenvalue greater if the smaller eigenvalue is smaller. In the case, when the both eigenvalues are similar, they can be smaller.

- Write a function `harris_response(img, σd , σi )` , which computes the response of Harris function for each pixel of input image *img*. For computation, use the derivative of Gaussian D_x, D_y with standard deviation σ_d and a Gaussian as window function with standard deviation σ_i .
- Write a function `keypoint_locations=harris(img, σd , σi ,thresh)` , which detects the maxima of Harris function which are greater than *thresh*, using standard deviation σ_d for derivatives and σ_i for the window function.
- Write a function `maximg=nms2d(response, threshold)` for non-maxima suppression in a 4D matrix response (i.e. take to consideration all 8 points in the 3×3 neigborhood and output non-zero only if the center is strictly larger than the neighborhood). The function returns a matrix of the same size as input matrix response, in which there will be response in places of local maxima which are greater than thresh and neighborhood and 0 elsewhere.

Scale-space - automatic scale estimation

The basic version of Harris or Hessian detector needs one important parameter: the scale on which it estimates gradients in the image and detects "blobs" or "corners". It can be shown that the scale can be estimated automatically for each image.

For this purpose, it is necessary to build the "scale-space" – a three dimensional space, in which two dimensions are *x,y*-coordinates in the image and the third dimension is the scale. The image is filtered to obtain its new versions corresponding to increasing scale. Suppressing the details simulates the situation when we are looking at the scene from greater distance.

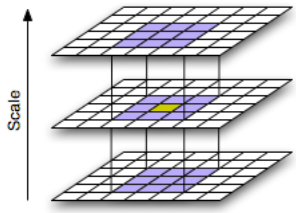
With increasing suppression of details the variance of intensities is decreasing. Therefore, selection of characteristic scale is necessary to have comparable gradients between two scales. The term "normalized derivative" (considering the "distance" between pixels) was introduced for this purpose to obtain a "scaleless" gradient.

$$\frac{\partial f}{\partial \xi} = \frac{\partial f}{\partial (x/\sigma)} = \sigma \frac{\partial f}{\partial x}, \quad N_x(x,y;\sigma) = \sigma D_x(x,y;\sigma), \quad N_{xx}(x,y;\sigma) = \sigma^2 D_{xx}(x,y;\sigma),$$

The normalized autocorrelation matrix is thus:

$$\mathbf{C}_{\mathbf{norm}}(x,y;\sigma_d,\sigma_i) = \sigma_d^2 G(x,y;\sigma_i) * \begin{bmatrix} D_x^2(x,y;\sigma_d) & D_x D_y(x,y;\sigma_d) \\ D_x D_y(x,y;\sigma_d) & D_y^2(x,y;\sigma_d) \end{bmatrix}$$

- Write a function `ss,sigma=create_scalespace(img,levels,step)`, which returns a 3D scale-space (matrix BxChxLxHxW, where H is height, W is width and L is number of *levels*), where `ss[:,:,0,:,:]` will be input image *img*, for which we will assume $\sigma_1 = 1$ and `ss[:,:,i,:,:]` will be it's filtered version $S(x,y,\sigma_{i+1})$, where $\sigma_{i+1} = \text{step} \cdot \sigma_i$ for $i \in \{1,\ldots,\text{levels}\}$. Vector *sigma* will contain values σ_i.
- Write a function `maximg=nms3d(response, threshold)` for non-maxima suppression in a 5D matrix *response* (i.e. take to consideration all 26 points in the neigborhood). The function returns a matrix of the same size as input matrix *response*, in which there will be *response* in places of local maxima which are greater than *thresh* and 0 elsewhere.



[6]

- Write a function `[hes,sigma]=scalespace_harris_response(img)`, which will use function `create_scalespace` and compute Harris response for normalized derivation of Gaussian.

Do not forget to multiply response function by the correct power of the corresponding sigma. Choose the parameters *step* and *levels* of function `create_scalespace` wisely (start with e.g. step=1.1, levels=30 and experiment with them). Normalize the derivatives with a standard deviation for each order of derivation (as described earlier). The result will be a 5D matrix *hes* with elements corresponding to normalized response of Hessian in scale-space.

Three different "sigmas" in this lab (do not mix them)

Symbol	Meaning	Used for	Where it appears in code
σ_d	**Derivative scale** (how wide is the derivative-of-Gaussian / gradient operator)	computing D_x, D_y	`harris_response(img, sigma_d, sigma_i)`
σ_i	**Integration scale** (window / averaging for second-moment matrix $\mathbf{M}$)	smoothing $I_x^2, I_x I_y, I_y^2$	`harris_response(img, sigma_d, sigma_i)`
σ	**Scale-space level** (how much the image is smoothed in the pyramid)	building $S(x,y,\sigma)$	`create_scalespace(img, levels, step)`

Common mistake: In scale-space Harris, the image at each level is already smoothed by σ. Therefore you must **NOT** apply another large σ_i or σ_d on top of it ("double blurring"). **Use small constant sigmas for the `harris_response` input, if run on the output of the `create_scalespace`.**

- Join functions `nms3d` and `scalespace_harris_response` to detector of Harris maxima with automatic scale estimation `keypoint_locations=scalespace_harris(img, thresh)`.

Return only points with response greater than *thresh*. The result will be matrix of size [N x 5] for position *b*, *ch*, *scale*, *y* and *x* of thresholded Harris maxima in the space-space. Choose internal *sigma*, so that visual results make sense. Use the same approach for *sigma_i* and *sigma_d* for `scalespace_harris`

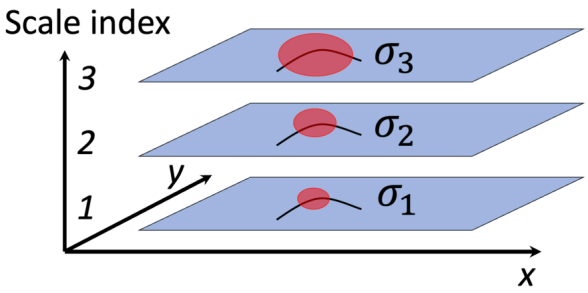
Top left point of the image has coordinates (0,0,0,0,0). You can use the following code for localization of points:

```
# get the matrix of maxima
nmsed = nms3d(response, thresh);

# find positions
```

```
    loc = torch.nonzero(nmsed)

    # Don't forget to convert scale index to scale value with use of sigma ...
```



torch.nonzero outputs indexes. For x, y indexes coincide with blob/corner center.
But scale index blob/corner size does NOT describe blob radius, we need to convert them to sigmas

[7]

For visualization of detected points use the function **visualize_detections**, defined in the local_detector.ipynb[8]:

Local feature detection[9].

To fulfil this assignment, you need to submit these files (all packed in one `.zip` file) into the upload system[10]:

- `local_detector.ipynb` - a notebook for data initialisation, calling of the implemented functions and plotting of their results (for your convenience, will not be checked).
- `local_detector.py` - file with the following methods implemented:
  - `harris_response` - function for computing response functions
  - `nms2d` , `nms3d` - functions for applying 2d and 3d non maxima supression
  - `harris` - function for getting coordinates of the single scale Harris maxima
  - `create_scalespace` - function for creating Gaussian scale space
  - `scalespace_harris_response` - function for computing response functions on scalespace tensor
  - `scalespace_harris` - function for getting coordinates of the multi scale Harris maxima
- `imageprocessing.py` - file with implemented functions from the previous assignment

Use template[11] of the assignment.

When preparing a zip file for the upload system, **do not include any directories**, the files have to be in the zip file root.

References

1. Overview of the most common used methods for feature point detection.[12]
2. Foundations of scale-space.[13]
3. Scale-space, the basic method for feature point detection indepented on the scale.[14]
4. Affine shape adaptation.[15]
5. Overview of the affine detectors.[16]

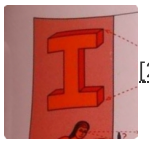
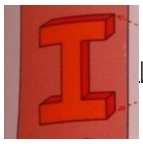
2. Computing local invariant description

The task of a detector is to reliably find feature points and their neighborhoods such that the detected points are covariant with a desired class of transformations (geometric or photometric). For instance, the basic version of Harris or Hessian detector detects points which are covariant with translation (if you shift the image, detected points shift the same amount). Hessian in the scale-space or DoG detector adds information about the scale of the region and therefore points are co-variant with similarity transformation up to an unknown rotation (detected points have no orientation assigned). Affine covariant detectors like MSER, Hessian-Affine or Harris-Affine are co-variant with affine transformations. The following text describes how to handle geometric information from the point neighbourhood and how to use it for normalization. Depending on the amount of information that a detector gives about a features, the descriptions can be invariant to translation, similarity, affine or perspective transformation.

Geometric normalization

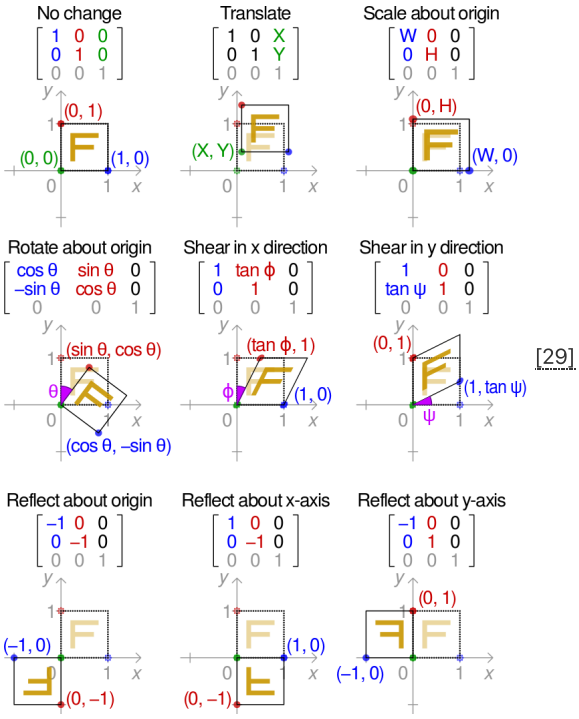
Geometric normalization is a process of geometric transformation of feature neighborhood into a canonical coordinate system. Information about geometric transformation will be stored in the form of “frame” – a projection of the canonical coordinate system into the neighborhood of a feature

point (or region) in the image. The frame will be represented by a 3×3 matrix A, the same as in the first lab[17]. The transformation matrix A is used to obtain a "patch" – a small feature neighborhood in the canonical coordinate system. All other measurements on this square patch of original image are now invariant to desired geometric transformation.

Original image	Translation	Translation and rotation	Similarity	Affine transformation
 [18]	 [19]	 [20]	 [21]	 [22]
 [23]	 [24]	 [25]	 [26]	 [27]

Example of geometric normalization, with different classes of transformation (author of images: Štepán Obdržálek).

Another example from Wikipedia[28]: Effect of applying various 2D affine transformation matrices on a unit square. Note that the reflection matrices are special cases of the scaling matrix.



[29]

For the construction of a transformation A, we use the geometric information about position and shape of point neighborhood:

1. For rotation- and translation-covariant transformation, we have coordinates *x,y* and angle α. The scale s (size of the frame) is fixed (manually) for all points.

$$A = \underbrace{\begin{bmatrix} s & 0 & x \\ 0 & s & y \\ 0 & 0 & 1 \end{bmatrix}}_{A_{norm}} \begin{bmatrix} cos(\alpha) & sin(\alpha) & 0 \\ -sin(\alpha) & cos(\alpha) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

2. For similarity-covariant point, we have coordinates *x,y* scale σ and angle α.

$$A = \underbrace{\begin{bmatrix} \sigma & 0 & x \\ 0 & \sigma & y \\ 0 & 0 & 1 \end{bmatrix}}_{A_{norm}} \begin{bmatrix} cos(\alpha) & sin(\alpha) & 0 \\ -sin(\alpha) & cos(\alpha) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3. For affine-covariant point, we can use three points to which the points from the canonical coordinate system are projected (as in the first lab[30]). We can also use a known position, partial affine transformation (a 2×2 submatrix) and the residual rotation (angle α) to construct the matrix A/

$$A = \underbrace{\begin{bmatrix} a_{11} & a_{12} & x \\ a_{21} & a_{22} & y \\ 0 & 0 & 1 \end{bmatrix}}_{A_{norm}} \begin{bmatrix} cos(\alpha) & sin(\alpha) & 0 \\ -sin(\alpha) & cos(\alpha) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Many detectors (all from the last lab) detect points (frames), which are similarity- or affine- covariant up to an unknown rotation (angle α). For instance, the position and scale give us a similarity co-variant point up to unknown rotation. Similarly, the matrix of second moments and the centre of gravity give us five constraints for an affine transformation and only the rotation remains unknown. To obtain a fully similarity- or affine-covariant point, we need to define orientation α from a partially geometrically normalized point neighborhood.

The normalization including orientation estimation has to be done in two steps. In the first step, the patch invariant to translation, scale and partial affine transformation is created. On this normalized patch, the dominant gradient is estimated. Both transformation are then used together in the transformation matrix A.

To estimate the orientations of the dominant gradients on the partially normalized regions (using A_{norm}), the gradient magnitudes and orientations are estimated for each region and a histogram of these gradients is computed. While we assume a random rotation of the region, it is necessary to compute gradients only in circular neighborhood. This can be done by weighting with a window-function (e.g. Gaussian) or with a condition like $(x - x_{center})^2 + (y - y_{center})^2 < (ps/2)^2$, where ps patch edge length. Otherwise the content of the corners would influence the orientation estimate undesirably. The orientations of gradients are weighted by their magnitudes. This means that a "stronger" gradient brings more to the corresponding bin of the histogram. For improved robustness, we can use linear interpolation to vote into the closest neighboring bins. At the end, the histogram is filtered with a 1D Gaussian and the maximum is found. For a more precise localization it is possible to fit a parabola into the maximum neighbourhood and to find the orientation with a precision over 360/(number of bins) degrees.

- Write a function `angle=estimate_patch_dominant_orientation(patch)` for dominant orientation estimation in a normalized patch. The input *img* is a partially normalized patch of the image (B x 1 PS x PS matrix of floats), the output is the *angle* measured from the *x*-axis of the image in clock-wise direction (angle for vector *(x,y)* can be computed with function `torch.atan2(y,x)`).

Now we are able to write the functions for geometric normalization of feature point neighborhoods with dominant orientations:

- for a known position, write a function `A,img_idxs=affine_from_location(keypoint_location)` , which output affine transformation *A* for patch extraction,*img* is the input image BxChXHxW, *keypoint_location* is [Bx5] is keypoint locations, as output from *scalespace_hessian* function.
- for a known position and orientation , write a function `A,img_idxs=affine_from_location_and_orientation(keypoint_location, orientation)` , where *img* is the input image BxChXHxW, *keypoint_location* is [Bx5] is keypoint locations, as output from *scalespace_hessian* function and *orientation* is [Bx1] angle in radians.

The output *A* and img_idx are input arguments for the function **extract_affine_patches** we implemented in the first lab.

Here's a pseudocode for normalization including orientation estimation:

```
A, img_idxs = affine_from_location(keypoint_locations)
patches =  extract_affine_patches(img, A, img_idxs, 19, 3.0)
ori = estimate_patch_dominant_orientation(patches)
A, img_idxs = affine_from_location_and_orientation(keypoint_locations, ori)
```

For the sake of clarity only the orientation of the strongest gradient is used for normalization in this task.

Affine covariance, Baumberg iteration

(optional study material for interested students)

The detection of similarity-covariant points, as maxima of Hessian in scale space, can be extended to affine-covariant points to an unknown rotation. It is based on the so called Baumberg iteration, the idea is to observe the distribution of gradients in the vicinity of the detected point. Assuming that there is a transformation, which "flattens" the intensity in the point's neighborhood (weighted average over the patch) in such a way that the distribution of gradients would be isotropic (with gradients distributed equally in all directions), the transformation can be found using the second moment matrix of gradients, i.e. the autocorrelation matrix of gradients that we already know.

$$\mathbf{C}(x, y; \sigma_d, \sigma_i) = G(x, y; \sigma_i) * \begin{bmatrix} D_x^2(x, y; \sigma_d) & D_x D_y(x, y; \sigma_d) \\ D_x D_y(x, y; \sigma_d) & D_y^2(x, y; \sigma_d) \end{bmatrix}$$

The formula above is given for the general case, when the calculation is done directly in the image and σ_i, σ_d corresponds to the feature scale. Because for the assignment you are working on the extracted patches, which already account for the feature scale, σ_i means a simple

average over the patch.

As we have said earlier, this matrix reflects the local distribution of gradients, we can show that when we find a transformation $\mu = \mathbf{C}^{-1/2}$, where $\mathbf{C}$ represents the whole patch, i.e. pooled over per-pixel C matrices by the window function. μ translates the coordinate system given by eigenvectors and eigenvalues of matrix $\mathbf{C}$ to a canonical one, the distribution of gradients in the transformed matrix μ will be "more isotropic". However, since we used a circular window function it is possible that for some gradients around, the weight will be calculated wrongly, or not at all. Therefore, it is necessary to repeat this procedure until the eigenvalues of matrix $\mathbf{C}_i$ from the i-th iteration of the point's neighborhood will be close to identity (a multiplication of identity). We'll find out by comparing the ratio of eigenvalue of matrix $\mathbf{C}_i$. The resulting array of local affine deformation is obtained as the total deformation of the neighborhood needed to "become" isotropic:

$$N = \prod_i \mu_i$$

This transformation leads (after the addition of translation and scaling) from the image coordinates to the canonical coordinate system. In practice, we are mostly interested in the inverse transformation $\mathbf{A} = \mathbf{N}^{-1}$.

- Write a function `affshape = estimate_patch_affine_shape (patch)` , which calculates one step of the Baumberg iteration for the patch. The function returns Bx3 ellipse parameters affshape **C**. The order is `[c11, c12, c22]` , as the **C** matrix is symmetrical.
- for a known position, orientation and affine shape , write a function `A,img_idxs=affine_from_location_and_orientation_and_affshape(keypoint_location, orientation, affshape)` , where *img* is the input image BxChXHxW, *keypoint_location* is [Bx5] is keypoint locations, as output from *scalespace_hessian* function, *orientation* is [Bx1] angle in radians, and affshape is [Bx3] matrix of ellipse parameters [a,b,c]. Hint: convert C to 2×2 submatrix A1 matrix by the following formula. Note, that square root and inversion are matrix expressions, not element-wise.

$$A1 = \sigma_i \, inv\left(C^{-1/2} / \sqrt{det(C^{-1/2})}\right)$$

.

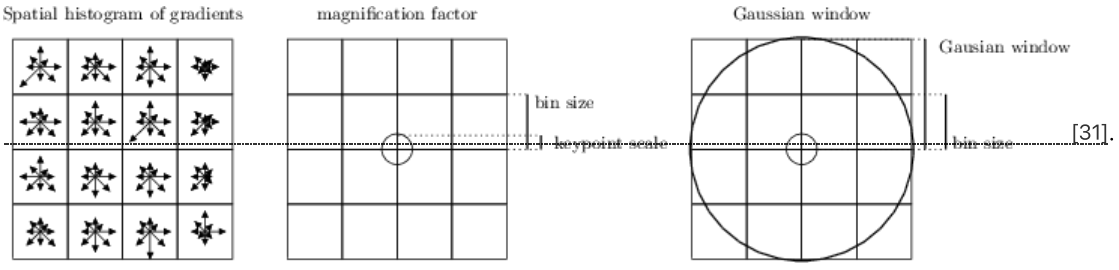
Photometric normalization

The goal of photometric normalization is to suppress the scene changes caused by illumination changes. The easiest way of normalization is to expand the intensity channel into whole intensity range. We can also expand the intensity channel such that(after transformation) the mean intensity is at half of the intensity range and the standard deviation of intensities <latex>\sigma</latex> corresponds to it's range (or better <latex>2\times\sigma</latex>). In case we want to use all three color channels, we can normalize each channel separately.

Computing the description

On the geometrically and photometrically invariant image patch from the last step, any low dimensional description can be computed. It is clear that the process of normalization is not perfect and therefore it is desirable to compute a description which is not very sensitive to the residual inaccuracies. The easiest description of the patch is the patch itself. Its disadvantages are high dimensionality and sensitivity to small translations and intensity changes. Therefore, we will try better descriptions too.

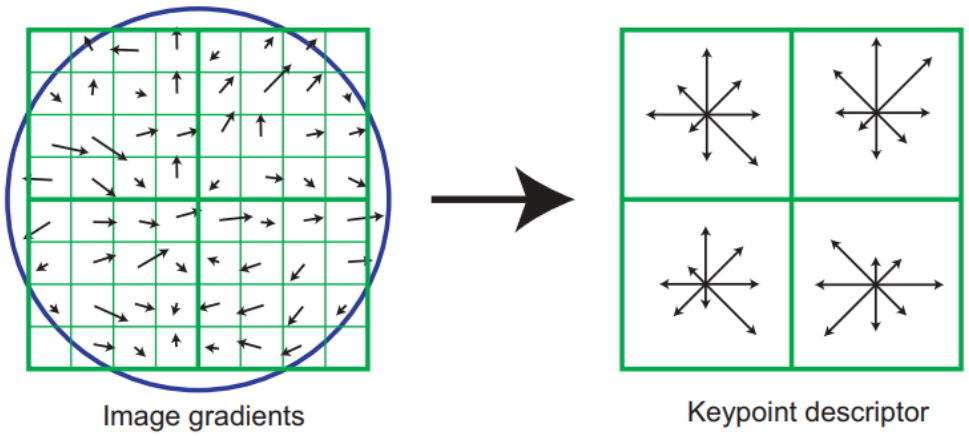
Write a function, `calc_sift_descriptor(input, num_ang_bins, num_spatial_bins, clipval)` which computes SIFT descriptor of the patch



[31].

Image source: VLFeat SIFT tutorial[32]

Here is a description from the original SIFT paper[33]

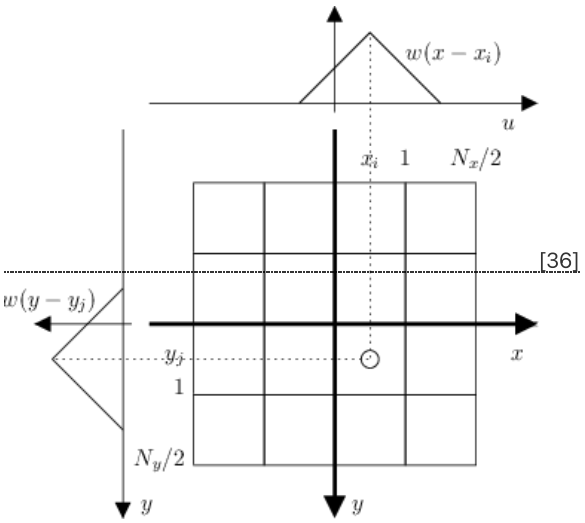


[34]

A keypoint descriptor is created by first computing the gradient magnitude and orientation (Hint: you can use `spatial_gradient_first_order` from the first lab) at each image sample point in a region around the keypoint location, as shown on the left. These are weighted by a Gaussian window, indicated by the overlaid circle. These samples are then accumulated into orientation histograms summarizing the contents over 4×4 subregions, as shown on the right, with the length of each arrow corresponding to the sum of the gradient magnitudes near that direction within the region.

This figure above shows a 2×2 descriptor array computed from an 8×8 set of samples, whereas the experiments in this paper use 4×4 descriptors computed from a 16×16 sample array. After this, descriptor is L2-normalized, maximum is clipped to `clip_val` and L2-normalized again.

Do not forget to weight the magnitudes by the distance to the subpatch center for the good performance, as shown below. Also, subpatches should be overlapping, see picture below from the https://www.vlfeat.org/api/sift.html[35]



[36]

What should you upload?

Local descriptors[37].

To fulfil this assignment, you need to submit these files (all packed in one `.zip` file) into the upload system[38]:

- `local_descriptor.ipynb` - a notebook for data initialisation, calling of the implemented functions and plotting of their results (for your convenience, will not be checked).
- `local_descriptor.py` - file with the following methods implemented:
  - `affine_from_location` - computes transformation matrix A which transforms point in homogeneous coordinates from canonical coordinate system into image from keypoint location (output of scalespace_harris or scalespace_hessian). Matrix A is the same, as in first assignment. Output of scalespace_hessian is of shape [N x 5], where N - is total number of detector features. And 5 is b_ch_sc_y_x, where b is image index in batch, ch - channel index (not used), sc - scale, y - height, x - width index.
  - `affine_from_location_and_orientation` - computes transformation matrix A which transforms point in homogeneous coordinates from canonical coordinate system into image from keypoint location (output of scalespace_harris or scalespace_hessian) and orientation
  - `estimate_patch_dominant_orientation` , - Function, which estimates the dominant gradient orientation of the given patches, in radians.
  - `calc_sift_descriptor` - functions for SIFT descriptor calculation

- Optional (for bonus points):
  - `affine_from_location_and_orientation_and_affshape` , - computes transformation matrix A which transforms point in homogeneous coordinates from canonical coordinate system into image from keypoint location (output of scalespace_harris or scalespace_hessian), orientation and affine_shape
  - `estimate_patch_affine_shape` - function, which estimates the patch affine shape by second moment matrix
- `imageprocessing.py` - file with implemented functions from the previous assignment
- `local_detector.py` - file with implemented functions from the previous assignment

Use template[39] of the assignment. When preparing a zip file for the upload system, **do not include any directories**, the files have to be in the zip file root.

Python and PyTorch Development [40]

References

1. David G. Lowe, "Distinctive image features from scale-invariant keypoints," International Journal of Computer Vision, 60, 2 (2004), pp. 91-110. [41]
2. A performance evaluation of local descriptors[42].
3. HPatches: A benchmark and evaluation of handcrafted and learned local descriptors[43]

3. Tentative correspondences and RANSAC

Preliminaries

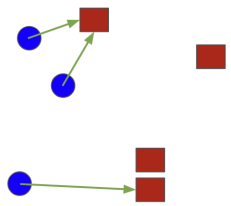
Before we start, we'll need a function `keypoint_locations, descs, A = detect_and_describe(img, det, th, affine, PS)` connecting the previously implemented functions together, such that it detects, geometrically and photometrically normalizes and describes feature points in the input image *img*. You can find function `detect_and_describe` in the this notebook[44].

Matching: tentative correspondences

After detection and description, which run on each image of the scene separately, we need to find correspondences between the images. The first phase is looking for the tentative correspondences (sometimes called “matches”). A tentative correspondence is a pair of features with descriptions similar in some metric. From now on, we will refer to our images of the scene as the left one and the right one. The easiest way is to establish tentative correspondences is to find all distances between the descriptions of features detected in the left and the right image. We call this table a “distance matrix”. We will compute the distance in Euclidean space. Let's try several methods for correspondence selection:

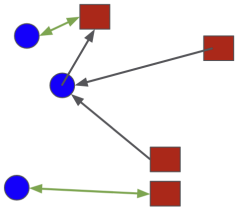
- **All** tentative correspondences without any filtering.

Nearest neighbor strategy. Features from img1 (blue circles) are matched to features from img2 (red squares). You can see, that it is asymmetric and allowing “many-to-one” matches.



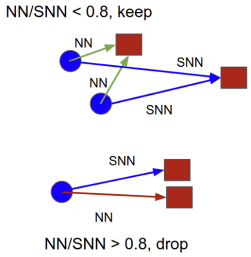
- Tentative correspondences of **mutually nearest** descriptions are created, when the nearest description in the left image for description A from right image is description B and at the same time, description A is the closest description in the right image to description B from the left image.

Mutual nearest neighbor strategy. Features from img1 (blue circles) are matched to features from img2 (red squares). Only cross-consistent matches (green) are retained.



- A different procedure is to find the two closest descriptions in the right image for each description from the left image and then test, if the ratio of **first/second closest** (**SNN**) description is smaller than a threshold (use e.g. 0.8). If so, the descriptions correspond and it is guaranteed, that there are no confusing descriptions which would be too close in the description space. This method can be used for well discriminative description - for instance SIFT. Write a function `matches_idxs, match_dists=match_snn(descs1, descs2, th)` , which implements this strategy.

Second nearest ratio strategy. Features from img1 (blue circles) are matched to features from img2 (red squares). For each point in img1 we calculate two nearest neighbors and check their distance ratio . If both are too similar (>0.8, bottom at Figure), then the match is discarded. Only confident matches are kept



- Combination of the two previous approaches: **mutual nearest neighbors, that also survive SNN check in both directions**.

The output of finding tentative correspondences are pairs of indices of features from the left and the right image with the closest descriptions.

First output will be a Nx2 matrix containing the indices of tentative correspondences, where the first row corresponds to indices in `descs1` and second row to indices in `descs2` . Second output will the correspondence quality Nx1 matrix: descriptor distance ratio for `match_snn` .

> Please, make sure to test your code for the corner-cases. Specifically, for situations, where the correct answer for matching would be “there is no match”.

Additional reading on matching strategies[45]

RANSAC and the geometric model of the scene

The tentative correspondences from the previous step usually contain many non-corresponding points, so called outliers, i.e. tentative correspondences with similar descriptions that do not correspond to the same physical object in the scene. The last phase of finding correspondences aims to get rid of the outliers. We will assume images of the same scene and search a model of the undergoing transformation. The result will be a model of the transformation and a set of the so called inliers, tentative correspondences that are consistent with the model.

The model of a planar scene

The relation between the planes in the scene is discussed in the course Digital Image[46], (look at the document on searching of the projective transformation[47]), or Geometry for Computer Vision[48]. In short:

The transformation between two images of a plane observed from two views (left and right image) using the perspective camera model is a linear transformation (called **homography**) of 3-dimensional homogeneous vectors (of corresponding points in the homogenoeous coordinates) represented by a regular 3×3 matrix $\mathbf{H}$:

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

For each pair of corresponding points $(x_i, y_i)^\top \leftrightarrow (x'_i, y'_i)^\top$ it holds:

$$\begin{aligned} x'(h_{31}\,x + h_{32}\,y + h_{33}) - (h_{11}\,x + h_{12}\,y + h_{13}) &= 0 \\ y'(h_{31}\,x + h_{32}\,y + h_{33}) - (h_{21}\,x + h_{22}\,y + h_{23}) &= 0 \end{aligned}$$

Let us build a system of linear constraints for each pair:

$$\underbrace{\begin{bmatrix} -x_1 & -y_1 & -1 & 0 & 0 & 0 & x'_1x_1 & x'_1y_1 & x'_1 \\ 0 & 0 & 0 & -x_1 & -y_1 & -1 & y'_1x_1 & y'_1y_1 & y'_1 \\ -x_2 & -y_2 & -1 & 0 & 0 & 0 & x'_2x_2 & x'_2y_2 & x'_2 \\ 0 & 0 & 0 & -x_2 & -y_2 & -1 & y'_2x_2 & y'_2y_2 & y'_2 \\ & \vdots & & & \vdots & & & \vdots & \\ -x_n & -y_n & -1 & 0 & 0 & 0 & x'_nx_n & x'_ny_n & x'_n \\ 0 & 0 & 0 & -x_n & -y_n & -1 & y'_nx_n & y'_ny_n & y'_n \end{bmatrix}}_{\mathbf{C}} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \\ h_{33} \end{bmatrix} = 0$$

This homogeneous system of equations has 9 degrees of freedom and we get one non-trivial solution as a right nullspace of the matrix $\mathbf{C}$. For a unique solution we need at least 8 linearly independent rows of matrix <latex>\mathbf{C}</latex>, i.e. 8 constraints from 4 corresponding points in a general position (no triplet of points can lie on a line!).

- implement a function `H=getH(min_sample)` that gets a matrix of 4 corresponding pairs of points and computes the homography $\mathbf{H}$ In case that some of the triplets will lie close to a line, it returns a **None** value. The input *u* is a 4×4 matrix as shown below, where x_i,y_i are point coordinates in the left image and x'_i,y'_i in the right image. :

$$u = \begin{bmatrix} x_1 & y_1 & x'_1 & y'_1 \\ x_2 & y_2 & x'_2 & y'_2 \\ x_3 & y_3 & x'_3 & y'_3 \\ x_4 & y_4 & x'_4 & y'_4 \end{bmatrix}$$

Do not forget to normalize by the bottomright element of the homography matrix, such that $H = [[h_{11}, h_{12}, h_{13}], [h_{21}, h_{22}, h_{23}], [h_{31}, h_{32}, 1]]$. You can use torch.linalg.svd[49].

- write a function `dist=hdist(H,pts_matches)` , which gets a Nx4 matrix *pts_matches* of N pairs of points in Euclidean coordinates and returns a vector *dist* (1xN) of squared distances of the points $\mathbf{H}(x_i, y_i, 1)^\top$ and $(x'_i, y'_i, 1)^\top$ in **Euclidean** space.

RANSAC

To find a correct model of the scene – a projective transformation between two planes, we need at least 4 inlier point pairs (not in a line). One way of picking such points in a robust way from all the pairs of acquired tentative correspondences is the RANSAC algorithm of M.Fischler and R.C.Bolles.

In our task, the RANSAC algorithm will have this form:

1. Uniformly sample 4 corresponding pairs. Write a function `sample(pts_matches, num)` , which takes Nx4 matrix and randomly selects num points out of it.
2. Compute the homography $\mathbf{H}$ out of the sampled point pairs.
3. Compute the support of the found model. If the transformation $\mathbf{H}$ was computed from an all-inlier sample, all inlier points from the left image (x_i, y_i) shall be projected by $\mathbf{H}$ to the corresponding points (x'_i, y'_i) in the other image. The support of the model is quantified by the number of points consistent with the model, i.e. the number of correspondences with distance (function `hdist`) of transformed point from the left image and the corresponding point in the right image smaller than a predefined threshold (e.g. 1-3 pixels).
4. If the support of the model is the biggest so far, we keep this so-far-the-best homography $\mathbf{H}^*$ and the corresponding set of inliers, i.e points that supported this model.
5. We iterate points 1-4 and keep track of so-far-the-best $\mathbf{H}^*$ transformation.

Set the number of iterations to 1000 in the beginning and debug the RANSAC algorithm on a simple pair of images. A more sophisticated stopping criterion is used in practice. It is based on the probability estimate of finding an all-inlier sample (of size 4 in our case) under the assumptions of iteratively estimated fraction of inliers. - Implement the stopping criterium is implemented function `new_max_iter = nsamples(n_inl, num_tc, sample_size, conf)` where *n_inl* is a number of inliers, *num_tc* is a number of tentative correspondences and *sample_size* is 4.

$$\text{new\_max\_iter} = \frac{\log(1 - \text{conf})}{\log(1 - \text{inl\_ratio}^{\text{sample\_size}})}$$

- use the provided functions to implement s function `Hbest,inl=ransac_h(pts_matches,threshold,confidence,max_iter)` that robustly estimates the homography $\mathbf{H}^*$ and a set of inliers *inl*. Input *pts_matches* is a Nx4 matrix of corresponding points in homogeneous coordinates (similar to function `hdist`). The parameter *threshold* is the maximum distance of reprojected points (computed via `hdist`) that will be marked as inliers (pairs consistent with the model $\mathbf{H}^*$). The parameter *confidence* is a value from the range (0,1), the requested confidence of getting a correct sample (a good value for testing is for example 0.95).
  The output is the best model $\mathbf{H}^*$ and an array inl (Nx1), where 0 represents an outlier and 1 represents an inlier.

What should you upload?

Local feature detection[50].

To fulfil this assignment, you need to submit these files (all packed in one `.zip` file) into the upload system[51]:

- `matching_and_ransac.ipynb` - a notebook for data initialisation, calling of the implemented functions and plotting of their results (for your convenience, will not be checked).
- `matching.py` - file with the following methods implemented:
  - `match_snn` - function for matching descriptors
- `ransac.py` - file with the following methods implemented:
  - `hdist` - function for reprojection error calculation
  - `sample` - function for random sampling correspondences for H or F calculation
  - `getH` - function which computes H from 4 correspondences
  - `nsamples` - function, which updates maximum number of iteration needed for given confidence level
  - `ransac_h` - function, which robustly estimates homography from noisy correspondences

Use template[52] of the assignment. When preparing a zip file for the upload system, **do not include any directories**, the files have to be in the zip file root.

Task related to the testing implemented functions on your own dataset will be added soon, please check updates

Testing

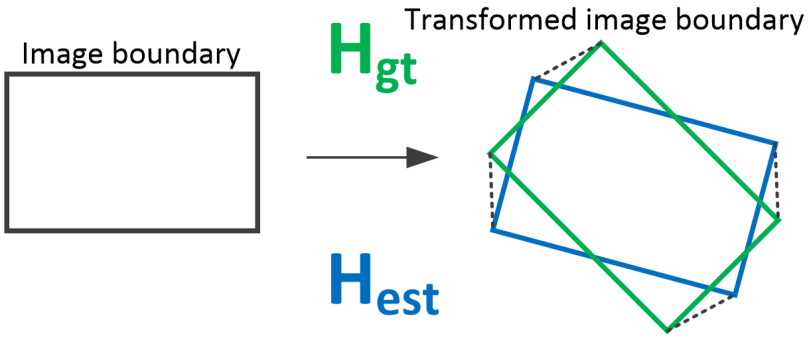
To test your code use the provided notebook notebook[53].

Tournament

In the tournament, you should implement a function `matchImages`, which takes two images (in form of torch.Tensor) and output homography transform, which maps coordinates in image 1 into image 2. All the parameters are up to participants. You are free in choosing the approach you would like to apply: integration and tuning the things you have done in the previous labs, or developing completely different algorithm, e.g. based on photometric error minimization. Important: you cannot use a solution from 3rd party library (by import or by copy-paste), but should develop it yourself.

Evaluation

Evaluation metric is mean average accuracy. Specifically, evaluation is done as following: Corner points (0,0), (0,h), (w, h), (w,0) are reprojected to the second image via estimated and ground truth homography. The average absolute error over 4 point is calclulated (MAE). See image below for the illustration. Image credit: Christiansen et.al, 2019, UnsuperPoint[54]



For accuracy threshold in **th** in [1.0, 2.0, 5.0, 10., 15., 20] percentage of image pairs, where **MAE < th** is calculated, resulting in average accuracy **Ai**. Finally, all per-threshold accuracies **Ai** are averaged into final score.

The number of image pairs in test dataset: around **40**. Timeout: **750 sec**. Rank start at 0 :) The number of points you get is **max(7, 17 - (your rank)/2)**. So, 1st place gets 17 total points (7 points mandatory and 10 bonus), 2nd place - 16.5 and so on. All participants get at least seven points if their solution produces non-zero accuracy.

If you submit after deadline, you cannot get the bonus points at all. The regular points (7) are still available - with standard penalty rules.

Local evaluation and debugging

For local evaluation you could use `get_MAE_imgcorners` function from the provided template[55]. We recommend to use Oxford-Affine dataset[56] for local validation purposes.

Important! Images are loaded with `img = kornia.image_to_tensor(cv2.imread(fname,0),False).float()/255.` , so their range is `[0,1.0]` , not `[0,255]`

What should you upload?

Wide baseline stereo tournament[57].

To fulfill this assignment, you need to submit these files (all packed in one `.zip` file) into the upload system[58]:

- `wbs.py` - file with all functions you need.
- `weights.pth` - Optional. If you have some trained CNN or another model, you could store parameters in this file

Use template[59] of the assignment. When preparing a zip file for the upload system, **do not include any directories**, the files have to be in the zip file root.

Remember that you are not allowed to use any 3rd party implementations of detectors, descriptors, matchers, RANSACs and so on. However, you may use utility functions, like spatial gradient, bluring, nms.

Hypertext Links

[1] https://gitlab.fel.cvut.cz/mishkdmy/mpv-python-assignment-templates/blob/master/assignment_0_3_correspondences_template/wide-baseline-stereo-demo.ipynb

[2] https://cw.fel.cvut.cz/wiki/_detail/courses/mpv/labs/2_correspondence_problem/wbs.png?id=courses%3Ampv%3Alabs%3A2_correspondence_problem%3Astart

[3] https://cw.fel.cvut.cz/wiki/_detail/courses/mpv/labs/2_correspondence_problem/akaze.png?id=courses%3Ampv%3Alabs%3A2_correspondence_problem%3Astart

[4] http://en.wikipedia.org/wiki/Corner_detection#The_Harris_.26_Stephens_.2F_Plessey_corner_detection_algorithm

[5] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=7ed2f0&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci%

[6] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=7c7d00&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci%

[7] https://cw.fel.cvut.cz/wiki/_detail/courses/mpv/labs/2_correspondence_problem/scalespace-index-800.png?id=courses%3Ampv%3Alabs%3A2_correspondence_problem%3Astart

[8] https://gitlab.fel.cvut.cz/mishkdmy/mpv-python-assignment-templates/blob/master/assignment_0_3_correspondences_template/local_detector.ipynb

[9] https://cw.fel.cvut.cz/b192/courses/mpv/labs/2_correspondence_problem/start

[10] https://cw.felk.cvut.cz/brute/

[11] https://gitlab.fel.cvut.cz/mishkdmy/mpv-python-assignment-templates

[12] http://en.wikipedia.org/wiki/Feature_detection_(computer_vision)

[13] http://www.bmia.bmt.tue.nl/education/courses/FEV/book/pdf/02%20Foundations%20of%20scale-space.pdf

[14] http://en.wikipedia.org/wiki/Scale-space

[15] http://en.wikipedia.org/wiki/Affine_shape_adaptation

[16] http://www.robots.ox.ac.uk/~vgg/research/affine/

[17] https://cw.fel.cvut.cz/wiki/courses/mpv/labs/1_intro/start#geometric_transformations_and_interpolation_of_the_image

[18] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=cf11cb&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci%

[19] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=7bd154&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci

[20] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=947566&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci

[21] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=c28af3&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci%

[22] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=58129f&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci%

[23] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=19669e&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci

[24] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=d7ed20&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci

[25] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=7e8997&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci
[26] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=9a9439&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci
[27] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=2ae1d6&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F2_hledani_korespondenci
[28] https://en.wikipedia.org/wiki/Affine_transformation
[29] https://cw.fel.cvut.cz/wiki/_detail/courses/mpv/labs/2_correspondence_problem/2d_affine_transformation_matrix.svg.png?id=courses%3Ampv%3Alabs%3A2_correspondence_problem%3Astart
[30] https://cw.fel.cvut.cz/wiki/courses/mpv/labs/1_intro/start#geometric_transformations_and_interpolation_of_the_image
[31] https://cw.fel.cvut.cz/wiki/_detail/courses/mpv/labs/2_correspondence_problem/sift-descr-easy.png?id=courses%3Ampv%3Alabs%3A2_correspondence_problem%3Astart
[32] https://www.vlfeat.org/api/sift.html
[33] https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf
[34] https://cw.fel.cvut.cz/wiki/_detail/courses/mpv/labs/2_correspondence_problem/sift-lowe.png?id=courses%3Ampv%3Alabs%3A2_correspondence_problem%3Astart
[35] https://cw.fel.cvut.cz/wiki/courses/mpv/labs/2_correspondence_problem/vlfeat_sift_reference
[36] https://cw.fel.cvut.cz/wiki/_detail/courses/mpv/labs/2_correspondence_problem/sift-can.png?id=courses%3Ampv%3Alabs%3A2_correspondence_problem%3Astart
[37] https://cw.fel.cvut.cz/b192/courses/mpv/labs/2_correspondence_problem/start
[38] https://cw.felk.cvut.cz/brute/
[39] https://gitlab.fel.cvut.cz/mishkdmy/mpv-python-assignment-templates
[40] https://cw.fel.cvut.cz/wiki/courses/mpv/labs/general_info
[41] http://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf
[42] http://lear.inrialpes.fr/pubs/2005/MS05/mikolajczyk_pami05.pdf
[43] https://arxiv.org/pdf/1704.05939
[44] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=82a23d&media=https%3A%2F%2Fgitlab.fel.cvut.cz%2Fmishkdmy%2Fmpv-python-assignment-templates%2Fblob%2Fmaster%2Fassignment_0_3_correspondences_template%2Fmatching_and_ransac.ipynb
[45] https://medium.com/@ducha.aiki/how-to-match-to-learn-or-not-to-learn-part-2-1ab52ede2022
[46] https://cw.fel.cvut.cz/wiki/courses/ae4m33dzo/start
[47] http://cmp.felk.cvut.cz/cmp/courses/383ZS/X383ZS2005-6_Leto/cvic3/geomtransf.pdf
[48] https://cw.fel.cvut.cz/b202/_media/courses/gvg/gvg-05-2021-03-15-1.pdf
[49] https://pytorch.org/docs/stable/torch.html#torch.linalg.svd
[50] https://cw.fel.cvut.cz/b192/courses/mpv/labs/2_correspondence_problem/start
[51] https://cw.felk.cvut.cz/brute/
[52] https://gitlab.fel.cvut.cz/mishkdmy/mpv-python-assignment-templates
[53] https://gitlab.fel.cvut.cz/mishkdmy/mpv-python-assignment-templates/blob/master/assignment_0_3_correspondences_template/matching_and_ransac.ipynb
[54] https://arxiv.org/abs/1907.04011
[55] https://gitlab.fel.cvut.cz/mishkdmy/mpv-python-assignment-templates/-/blob/master/wbs_tournament_template/wbs.py
[56] http://www.robots.ox.ac.uk/~vgg/research/affine/
[57] https://cw.fel.cvut.cz/b192/courses/mpv/labs/2_correspondence_problem/start
[58] https://cw.felk.cvut.cz/brute/
[59] https://gitlab.fel.cvut.cz/mishkdmy/mpv-python-assignment-templates